Rapport de Travaux Pratiques : Modélisation et Programmation sur ABB RobotStudio

ABOURICH Ayoub

5ème année Génie Électrique
Option : Automatique, Robotique, Informatique Industrielle
EMG Rabat

Encadré par :
Pr. BEKRI Reda

21 février 2026

Table des matières

1 Introduction **3**

2 TP 1 : Création de la station et des trajectoires **3**
2.1 Initialisation de la station et ajout des équipements 3
2.2 Création des cibles (Targets) et de la trajectoire 3
2.3 Configuration des paramètres de mouvement 4

3 TP 2 : Programme RAPID Simple **5**
3.1 Exploration du code généré 5
3.2 Modification et automatisation du code 6

4 Conclusion **7**

1 Introduction

Ce rapport détaille les étapes réalisées lors des travaux pratiques sur le logiciel de simulation ABB RobotStudio. L'objectif est de se familiariser avec l'environnement de modélisation 3D, la création de cibles et de trajectoires, ainsi que la configuration et la modification du code en langage RAPID pour automatiser les tâches du robot.

2 TP 1 : Création de la station et des trajectoires

2.1 Initialisation de la station et ajout des équipements

La première étape consiste à mettre en place l'environnement de travail virtuel :

— Création d'un nouveau projet en sélectionnant l'option *Solution avec station et système de commande virtuel*.

— Importation du modèle de robot industriel **IRB 120_3kg_0.58m** depuis les bibliothèques ABB.

— Ajout d'un outil de travail nommé `MyTool` à partir de la bibliothèque des équipements.

— L'outil a ensuite été glissé sur le robot dans l'arborescence afin de l'attacher physiquement à la bride de ce dernier.

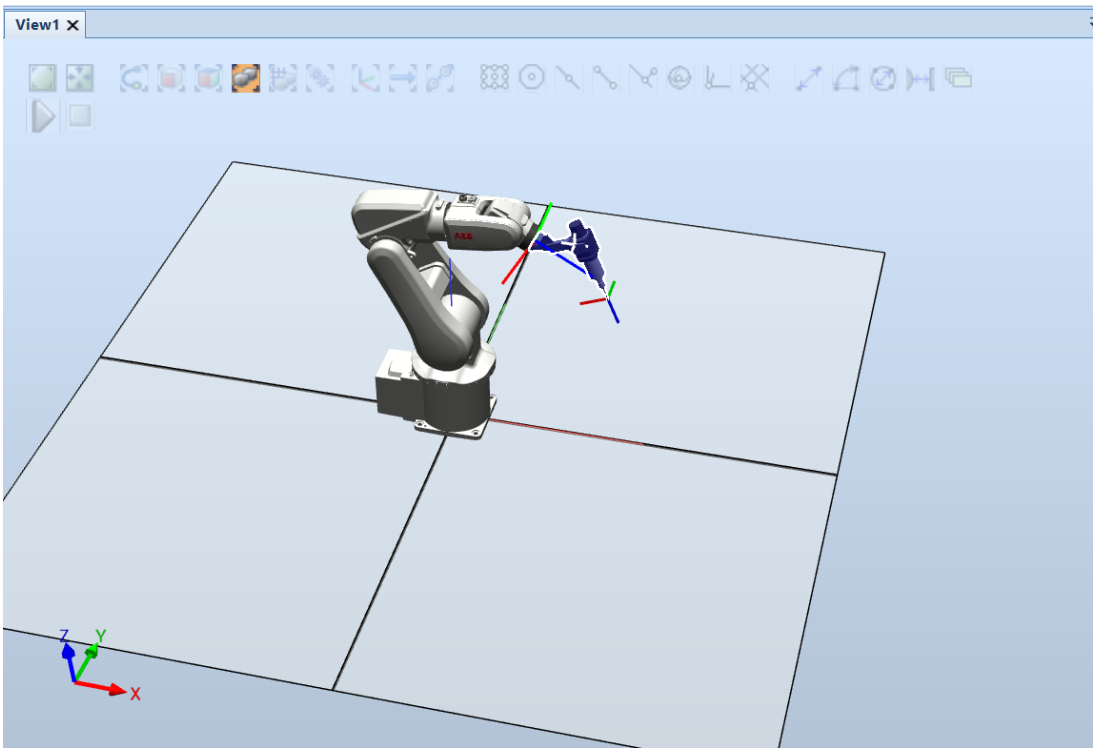

FIGURE 1 – Environnement RobotStudio avec le robot IRB 120 et l'outil MyTool attachés.

2.2 Création des cibles (Targets) et de la trajectoire

Une fois le manipulateur équipé, il a fallu définir les points de passage (cibles) :

— Choix du mode de déplacement manuel linéaire pour positionner l'effecteur.

— Création et enregistrement de quatre cibles distinctes (`Target_10`, `Target_20`, `Target_30`, `Target_40`) formant un contour fermé, puis vérification de leur présence dans l'arborescence.
— Ajout d'une nouvelle trajectoire (nommée `Path_10`) et intégration des cibles en les faisant glisser dans l'ordre souhaité.

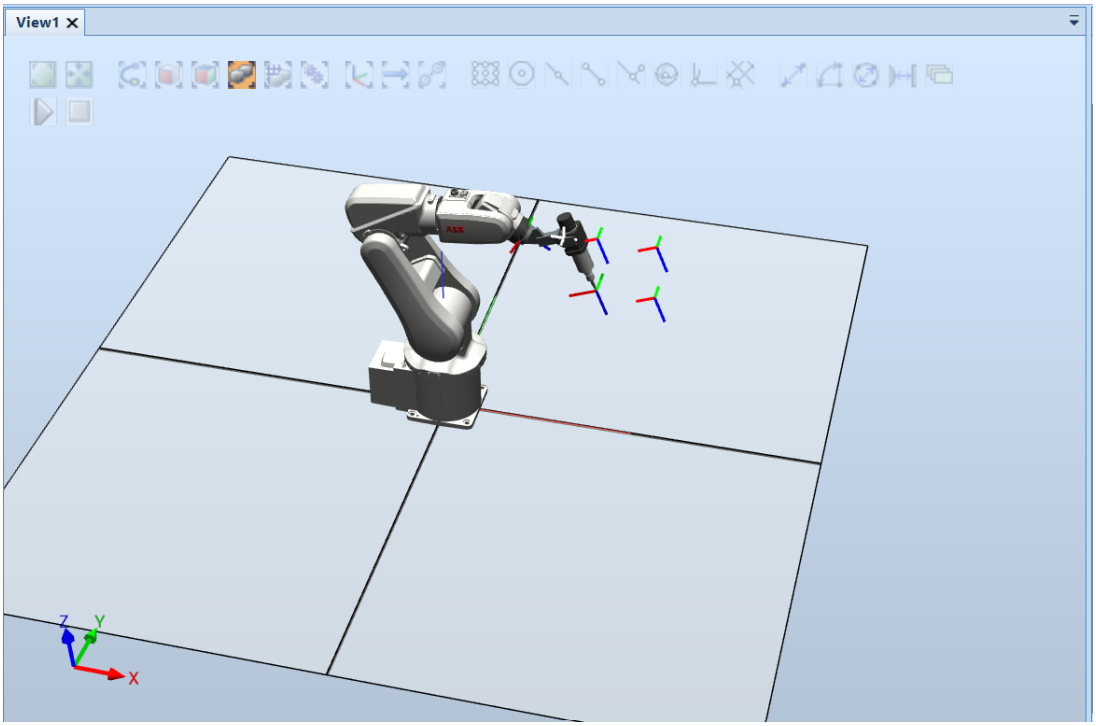

FIGURE 2 – Création des cibles formant un contour et génération de la trajectoire.

2.3 Configuration des paramètres de mouvement

Les instructions de mouvement par défaut ont été ajustées pour optimiser la cinématique :

— Sélection de toutes les cibles de la trajectoire pour modifier les arguments des instructions.
— Réglage de la zone d'approche sur la valeur *fine* pour s'assurer que le robot atteint exactement chaque point.
— Paramétrage de la vitesse de déplacement (ex : 30 m/s ou `v1000`).
— Calcul des configurations d'axes pour vérifier l'accessibilité de la trajectoire.
— Le type de mouvement a également été modifié de Linéaire (`MoveL`) à Articulaire (`MoveJ`) pour observer la différence de comportement cinématique.

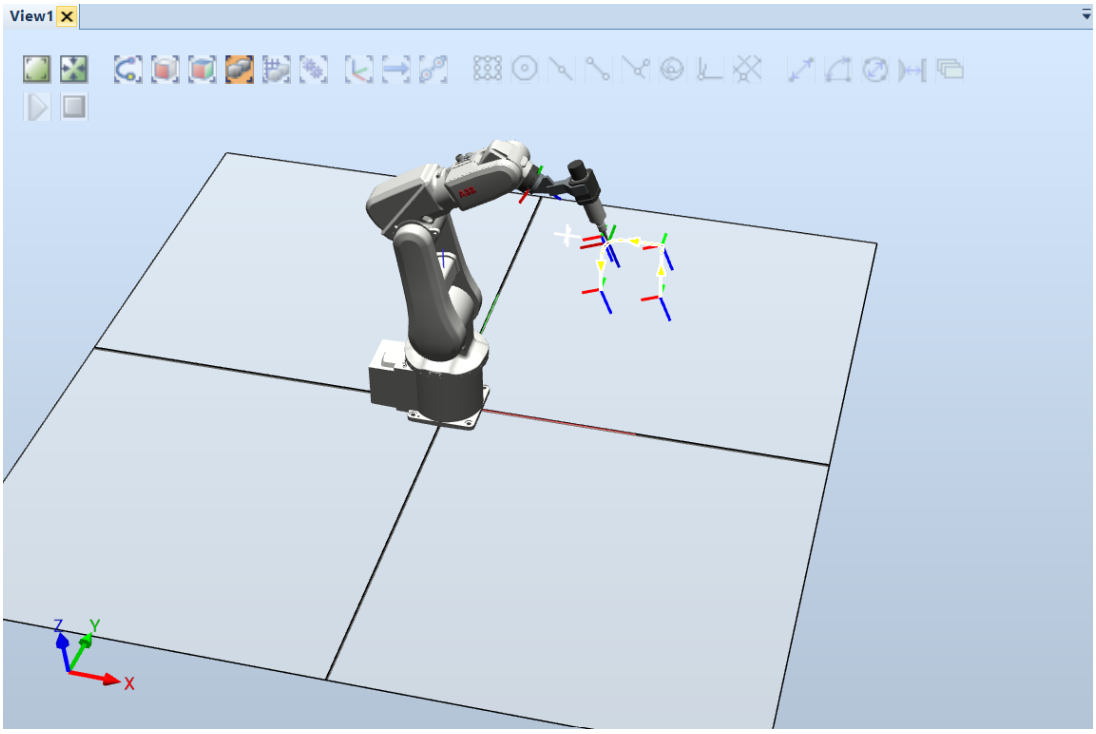

FIGURE 3 – Résultat du TP 1 : Exécution et simulation de la trajectoire en boucle fermée.

3 TP 2 : Programme RAPID Simple

3.1 Exploration du code généré

La seconde partie du TP se concentre sur l'interaction avec le contrôleur via le code source :

- Accès à l'onglet **RAPID** de RobotStudio pour visualiser le programme vide puis le code généré automatiquement.
- Navigation vers le module principal (`Module1`) et recherche de la routine `main`.
- Observation des déclarations : les cibles sont définies comme des constantes de type `robtarget` contenant les coordonnées cartésiennes et les quaternions d'orientation. La trajectoire est stockée dans la procédure `PROC Path_10()`.

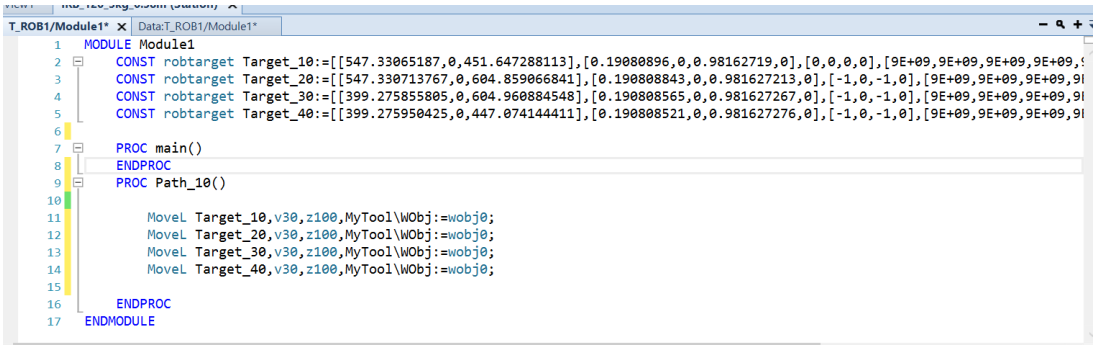

FIGURE 4 – Interface de l'éditeur RAPID affichant les cibles et les instructions de mouvement.

3.2 Modification et automatisation du code

Pour automatiser le fonctionnement de la station, le code a été modifié manuellement :

— Déclaration d'une variable de type entier nommée `compteur` dans la section `VAR`.
— Intégration d'une boucle `WHILE` dans la routine principale pour répéter l'exécution de la trajectoire.
— Ajout de l'incrémentation du compteur à chaque itération de la boucle pour gérer la condition d'arrêt.
— Sauvegarde, application des modifications au contrôleur, puis démarrage de la simulation finale.

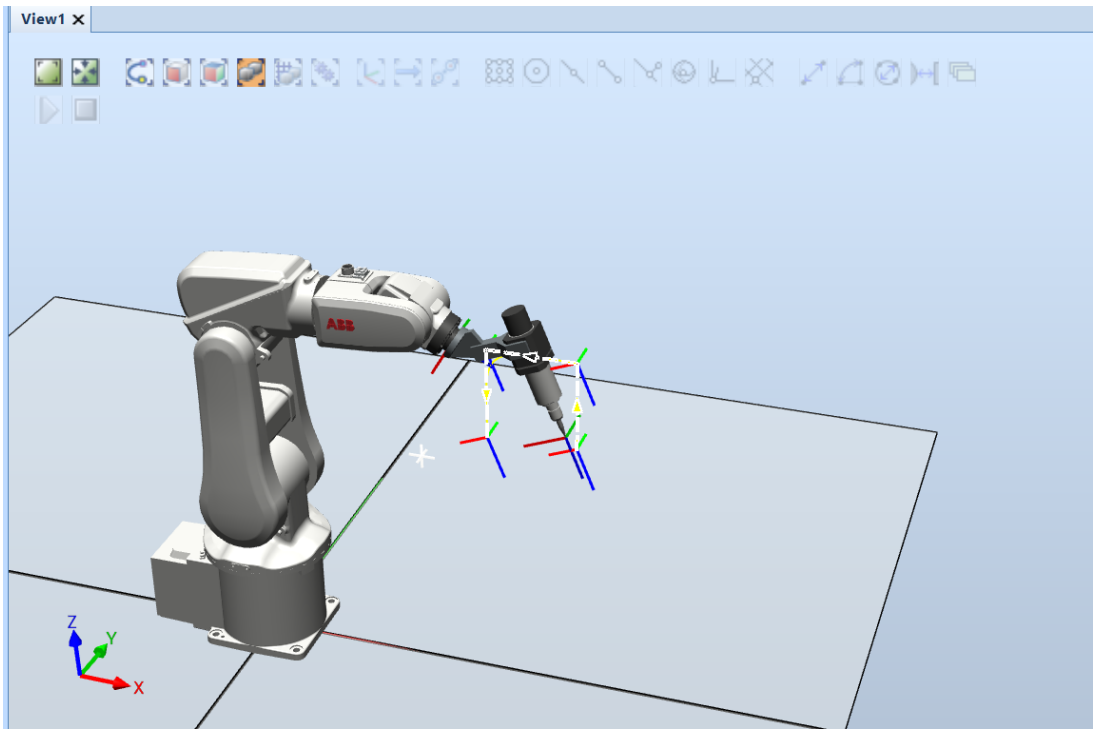

FIGURE 5 – Résultat du TP 2 : Simulation avec boucle d'automatisation codée en RAPID.

4 Conclusion

Ces manipulations successives ont permis d'appréhender le flux de travail complet en robotique industrielle simulée : depuis la définition de la cellule de travail, l'apprentissage des points, jusqu'à l'ajustement dynamique de la programmation sous-jacente en langage RAPID.